Temporal load shifting and provider capacity balancing in fixed-capacity inference services: a finite-game simulation study

Anonymous Author

Abstract

Time-of-use pricing can move flexible computing demand, but it can also redirect requests between providers. These effects are often combined in one peak-utilisation metric. This paper develops a simulation model that separates temporal load shifting from provider capacity balancing in a fixed-capacity inference market. The model includes two capacity-heterogeneous providers, an application programming interface intermediary, direct access, and time-rigid and time-flexible requests. Origin–destination flows conserve every request before users choose a channel. Routing and quality of service (QoS) are solved as a joint fixed point. The intraday load shape is taken from 1,429,737 BurstGPT records, while controlled vLLM measurements calibrate the QoS function. Provider competition is evaluated on a 225-point linear price-shape grid. A restricted bimatrix equilibrium is solved with Nashpy and checked against every unilateral grid deviation. The resulting finite game has a pure profile with zero grid regret. Relative to the uniform-price equilibrium, dynamic pricing reduces aggregate peak load by 13.03%, reduces maximum provider utilisation by 17.08%, and raises minimum QoS from 0.888 to 0.976. Across nine fully re-solved scenarios, it continues to reduce the peak and maximum utilisation and to improve minimum QoS. System profit changes from −37.43% to +9.41%, so its effect changes sign. A 1,024-point Latin-hypercube diagnostic finds no profitable off-grid deviation, but does not prove a continuous-space equilibrium. The results support time-of-use pricing as a QoS management mechanism under the modelled conditions.

Keywords: simulation modelling; inference services; time-of-use pricing; demand response; quality of service; finite games; verification and validation

1 Introduction

Online large language model (LLM) services operate with finite graphics processing unit (GPU) capacity. Their load changes during the day, while short-run capacity cannot be adjusted at the same speed. A local overload can increase time to first token (TTFT), queueing delay, and failed service-level objectives. Serving systems address these problems through batching, memory management, scheduling, and disaggregation [1, 2, 3, 4, 5]. Those methods act after requests reach the platform.

Pricing offers a different control. A provider can charge more during busy periods and less when capacity is underused. Flexible requests may then move in time. This idea follows a long line of electricity demand-response research [6, 7, 8, 9, 10]. The analogy is useful because both systems face short-run capacity limits and variable demand. It is incomplete because inference requests can also switch providers or purchase channels.

That distinction matters for measurement. A lower maximum provider utilisation does not necessarily mean that the total market peak has fallen. Demand may simply move from a small provider to a larger one. Movement between providers is a spatial capacity-balancing effect, whereas movement between periods is a temporal load-shifting effect. A model that reports only the largest provider-period utilisation cannot identify which mechanism produced a QoS gain.

Recent work has studied inference costs, menu pricing, token contracts, and Stackelberg routing games [11, 12, 13, 14, 15]. A separate systems literature provides detailed serving simulators and

workload measurements [16, 17]. However, a market simulation still needs an explicit account of native arrival time, time migration, channel choice, routing, congestion, and payment. Without that account, a reported load shift may reflect market growth, exit, or provider substitution rather than rescheduling.

This study asks whether linear time-of-use price shapes can protect QoS when these mechanisms are modelled separately. The model fixes total demand and represents time movement with origin–destination (OD) flows. Users choose a channel only after reaching a destination period. The application programming interface (API) intermediary then routes its traffic according to wholesale prices and provider QoS. Provider load determines QoS, which feeds back into channel choice and routing.

The study makes three contributions. First, it introduces a conserved-demand simulation that distinguishes aggregate peak load from a provider hotspot. Second, it places the demand model inside a verified routing–QoS fixed point and a finite provider game. Third, it evaluates the mechanism with a real load-shape anchor, a measured QoS-shape anchor, a complete 225-point deviation scan, an off-grid diagnostic, and nine fully re-solved parameter scenarios. Across these experiments, peak load and maximum provider utilisation fall and minimum QoS rises. The profit effect changes sign, and the finite-game result is not a continuous-space Nash claim.

2 Related research

2.1 Inference serving, workload measurement, and market models

Inference-serving research has concentrated on decisions inside the computing system. Clipper and Clockwork study predictable low-latency inference [18, 19]. Orca, vLLM, Sarathi-Serve, Splitwise, and DistServe improve batching, memory use, or phase scheduling for generative models [1, 2, 3, 4, 5]. Vidur adds a large-scale simulation framework for LLM serving [16]. These studies establish that latency and goodput depend strongly on local load. They generally treat demand as an input rather than a price-responsive market outcome.

Workload datasets improve the empirical basis of serving studies. BurstGPT contains real Azure OpenAI request records and reports bursty arrival and token patterns [17]. The present paper uses this dataset for its intraday token-load shape. It does not infer price sensitivity or migration cost from the trace because the trace contains no experimental price variation.

Market studies address costs and strategic decisions. Work on the market for intelligence and LLM menu pricing examines supply, demand, and contract design [11, 12]. PriLLM formulates a data-calibrated Stackelberg routing and pricing game [13]. Other recent models examine GPU platform pricing and token allocation [14, 15]. These papers move beyond scheduling, but they do not provide the same OD separation between time movement and provider switching used here.

2.2 Electricity pricing as the modelling foundation

Electricity pricing provides the closest established model of price-induced temporal demand response. Spot and real-time pricing link scarcity to time-varying prices [6, 7]. Demand-response surveys describe the operational requirements and the wide variation in user response [8, 10]. These studies show that a time-varying price can alter peak demand, but the realised change depends on user flexibility and tariff design.

Empirical results also distinguish conservation from rescheduling. Faruqui and Sergici review 15 household pricing experiments and report peak reductions of 3–6% for time-of-use tariffs and 13–20% for critical-peak pricing [9]. Allcott finds reduced peak electricity use under hourly pricing without a corresponding increase in average off-peak use [20]. These findings show why a lower peak should not automatically be called a time shift. The present simulation makes the distinction observable through OD flow conservation.

Utility and game models provide the mathematical basis for linking prices and demand. Samadi et al. derive real-time prices from utility maximisation, while Yang et al. formulate time-of-use

electricity pricing as a game [21, 22]. Stackelberg formulations place a utility or retailer above responding users [23, 24]. These models establish a useful decision sequence. They usually have one seller and a physical supply-balance constraint. The inference market considered here has competing providers, an intermediary, direct access, and endogenous QoS.

2.3 Congestion pricing, discrete choice, and finite games

Congestion pricing and revenue management explain how prices allocate limited capacity over time [25, 26, 27, 28]. Discrete-choice models provide a standard basis for price-sensitive alternatives [29, 30, 31]. Platform models clarify why an intermediary can change allocation between users and providers [32]. The present simulation combines these elements, but restricts attention to a small and reproducible market.

The provider game is finite because each linear price shape is evaluated on a specified parameter grid. Every finite game has a mixed Nash equilibrium [33, 34]. Nashpy implements support enumeration and Lemke–Howson methods for bimatrix games [35, 36]. A finite equilibrium does not establish equilibrium in the unrestricted continuous price space. This paper reports both full-grid regret and a separate Latin-hypercube deviation diagnostic.

Table 1: Relationship between prior research and the present simulation.

Research stream	Main result used here	Element added in this study	Main limit
Electricity demand response	Prices can alter peak demand, with heterogeneous response.	Conserved OD flows distinguish time movement from provider switching.	Behavioural parameters are not estimated from inference users.
Inference serving systems	QoS changes nonlinearly near capacity.	QoS enters channel choice, routing, payment, and provider load.	The measurement anchor uses one GPU and two small models.
LLM pricing and routing	Capacity, price, and routing should be solved together.	Two providers compete while an intermediary best-responds.	Strategies remain a finite price-shape family.
Finite-game methods	Restricted bimatrix equilibria can be computed and checked.	Every provider candidate is scanned for unilateral deviation.	Off-grid sampling is diagnostic rather than a proof.

Table 1 states the paper's position. Its contribution is not a new electricity tariff or a new serving scheduler. It is a simulation design that separates two price-response mechanisms and states what each result can support.

3 Methodology

3.1 Market scope and decision sequence

A day is divided into periods $t \in \mathcal{T} = \{1, \ldots, T\}$ of length h. Two providers $m \in \mathcal{M} = \{A, B\}$ have fixed capacities $G_A > G_B$. Users can buy directly from either provider or through one API intermediary. The channel set is $\mathcal{C} = \{I, A, B\}$, where I denotes the intermediary. Total demand is fixed, and no outside option or market-growth term is used in the final model.

Providers choose wholesale and direct price shapes simultaneously. The intermediary observes these prices and chooses its retail price shape and routing parameter from a finite response set. Given these decisions, demand, routing, provider load, and QoS are solved together. Figure 1 shows the market and the numerical workflow.

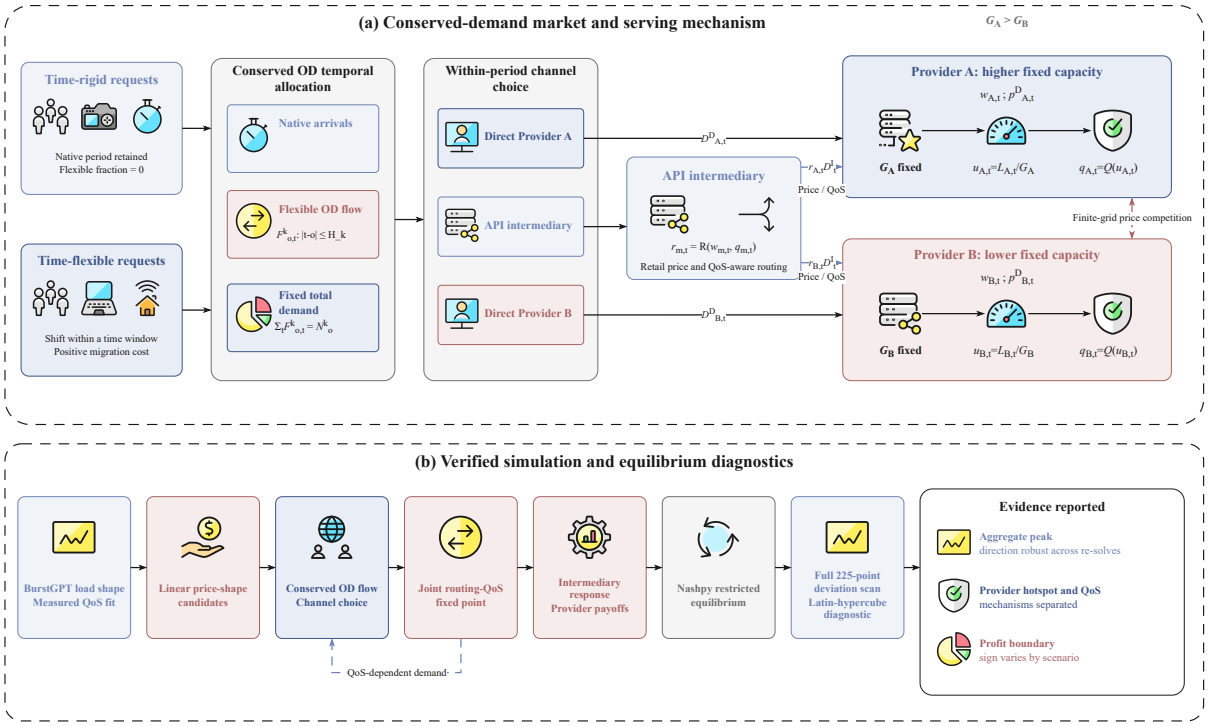


Figure 1: Conserved-demand market and numerical workflow. Panel (a) separates native arrivals, OD time movement, within-period channel choice, intermediary routing, and provider congestion. Solid arrows represent requests or routed traffic. Dashed arrows represent price and QoS signals. Panel (b) shows the data anchors, finite price candidates, joint fixed point, bimatrix solution, complete deviation scan, and reported evidence. Icons are from Streamline Ultimate Color under CC BY 4.0.

3.2 Native demand and conserved temporal movement

There are two request types, $k \in \{R, F\}$. Each type has native demand $N_{k,o}$ in origin period o. The rigid type has zero flexible fraction. The flexible type can move within a declared window H_k. Let $\phi_k \in [0, 1]$ denote the flexible fraction and κ_k the migration cost.

The deterministic utility of channel c in destination period t is

$$V_{k,c,t} = \log b_c - \alpha_k p_{c,t} - \omega_q(1 - q_{c,t}), \tag{1}$$

where b_c is a channel preference, α_k is price sensitivity, and $q_{c,t}$ is channel QoS. The channel prices are $p_{I,t} = p^I_t$, $p_{A,t} = p^D_{A,t}$, and $p_{B,t} = p^D_{B,t}$. The inclusive destination value is

$$S_{k,t} = \log\left(\sum_{c \in \mathcal{C}} \exp(V_{k,c,t})\right). \tag{2}$$

For t inside the migration window of origin o, the flexible destination probability is

$$\pi_{k,o,t} = \frac{\exp(S_{k,t} - \kappa_k|t - o|)}{\sum_{s:\,|s-o| \le H_k} \exp(S_{k,s} - \kappa_k|s - o|)}. \tag{3}$$

The OD flow is then

$$F_{k,o,t} = (1 - \phi_k)N_{k,o}\mathbf{1}\{t = o\} + \phi_k N_{k,o}\pi_{k,o,t}. \tag{4}$$

Equation (4) gives

$$\sum_{t \in \mathcal{T}} F_{k,o,t} = N_{k,o} \quad \text{and} \quad \sum_{c,t} D_{c,t} = \sum_{k,o} N_{k,o}. \tag{5}$$

Because total demand is conserved, a lower aggregate peak cannot be produced by exit or market contraction.

3.3 Channel choice, routing, and QoS

After reaching destination period t, users choose a channel with share

$$s_{k,c,t} = \frac{\exp(V_{k,c,t})}{\sum_{d\in\mathcal{C}} \exp(V_{k,d,t})}. \tag{6}$$

Destination demand and channel demand are

$$\widetilde{D}_{k,t} = \sum_{o} F_{k,o,t}, \qquad D_{c,t} = \sum_{k} s_{k,c,t}\widetilde{D}_{k,t}. \tag{7}$$

The intermediary routes its demand by wholesale price and QoS:

$$r_{m,t} = \frac{\exp(-\beta w_{m,t} + \eta q_{m,t})}{\sum_{j\in\mathcal{M}} \exp(-\beta w_{j,t} + \eta q_{j,t})}, \qquad \sum_{m} r_{m,t} = 1. \tag{8}$$

The intermediary channel QoS is $q_{I,t} = \sum_m r_{m,t} q_{m,t}$. Direct channels use the corresponding provider QoS.

Provider load and utilisation are

$$L_{m,t} = D_{m,t} + r_{m,t} D_{I,t}, \qquad u_{m,t} = \frac{L_{m,t}}{G_m}. \tag{9}$$

The QoS function is the same function fitted to the vLLM measurements:

$$q_{m,t} = Q(u_{m,t}) = \exp\left[-\zeta \max(u_{m,t} - \bar{u}, 0)^2\right]. \tag{10}$$

Equations (1)–(10) form a joint fixed point in $(\boldsymbol{q}, \boldsymbol{r})$. The implementation starts from unit QoS and equal routing, applies damping $\lambda = 0.35$, and allows at most 200 iterations. It stops when

$$\max\left\{\|\boldsymbol{q} - Q(\boldsymbol{u})\|_\infty, \|\boldsymbol{r} - R(\boldsymbol{w}, \boldsymbol{q})\|_\infty\right\} \le 10^{-9}. \tag{11}$$

Proposition 1 (Existence of a market fixed point)**.** *For fixed prices, finite native demand, and positive capacities, the market mapping has at least one fixed point in* $[0,1]^{2T} \times \Delta(\mathcal{M})^T$*, where* $\Delta(\mathcal{M}) = \{\boldsymbol{r} \in \mathbb{R}^2_+ : r_A + r_B = 1\}$.

Proof. The channel and routing shares are continuous softmax functions. The OD probabilities are continuous on their finite windows. The QoS function in Eq. (10) is also continuous at $u = \bar{u}$. Together, these functions map the compact convex set $[0,1]^{2T} \times \Delta(\mathcal{M})^T$ into itself. Brouwer's fixed-point theorem guarantees at least one fixed point. This argument does not imply uniqueness or convergence of every numerical iteration. □

3.4 Linear price shapes and accounting

The observed load shape is centred and scaled as $\ell_t \in [-1, 1]$. Provider m chooses

$$w_{m,t} = \mathrm{clip}(\bar{w}_m(1 + \delta^w_m \ell_t), \underline{w}, \overline{w}), \tag{12}$$

$$p^D_{m,t} = \mathrm{clip}\big(\bar{p}^D_m(1 + \delta^D_m \ell_t), \underline{p}, \overline{p}\big). \tag{13}$$

The intermediary retail price has the same linear form. The grid search discretises the four parameters $(\bar{w}_m, \delta^w_m, \bar{p}^D_m, \delta^D_m)$, not the eight period prices. This choice keeps the policy interpretable while avoiding derivatives through a non-smooth best response and a congestion fixed point.

Provider profit is

$$\begin{aligned}\Pi_m = {} & h\sum_t w_{m,t} r_{m,t} D_{I,t} q_{m,t} + h\sum_t p^D_{m,t} D_{m,t} q_{m,t} \\ & - hTc_G G_m - hc_q \sum_t (1 - q_{m,t})(D_{m,t} + r_{m,t} D_{I,t}).\end{aligned} \tag{14}$$

The intermediary profit is

$$\Pi_I = h\sum_t p_t^I D_{I,t} q_{I,t} - h\sum_{m,t} w_{m,t} r_{m,t} D_{I,t} q_{m,t} - hc_q \sum_t (1 - q_{I,t}) D_{I,t}. \tag{15}$$

The factor q in revenue represents completed service. The degradation term represents a separate operating or SLA cost attached to incomplete quality. This is a conservative accounting choice rather than a claim about a specific provider contract.

Proposition 2 (Internal transfer conservation)**.** *The wholesale settlement cancels exactly from system profit* $\Pi_{\text{sys}} = \Pi_A + \Pi_B + \Pi_I$.

Proof. For each provider and period, Eq. (14) contains $+hw_{m,t}r_{m,t}D_{I,t}q_{m,t}$ and Eq. (15) contains the same term with a negative sign. Summing the three profits removes every wholesale transfer. □

3.5 Finite provider game and equilibrium certificate

Each provider has the candidate set

$$\mathcal{S}_m = \{0.27, 0.575, 0.88\} \times \{-0.4, -0.2, 0, 0.2, 0.4\} \times \{0.6, 1.1, 1.6\} \times \{-0.4, -0.2, 0, 0.2, 0.4\}, \tag{16}$$

which contains 225 linear price shapes. Wholesale prices are clipped to $[0.25, 0.90]$, while direct and retail prices are clipped to $[0.45, 2.10]$. The intermediary candidate set is

$$\mathcal{B} = \{0.8, 1.1, 1.5\} \times \{-0.3, 0, 0.3\} \times \{1.5, 4.0\}, \tag{17}$$

where the entries are retail base, retail slope, and routing price sensitivity β. Uniform pricing restricts all slopes to zero.

For a provider pair (i, j), the intermediary response is

$$b(i,j) \in \arg\max_{b \in \mathcal{B}} \Pi_I(i, j, b), \tag{18}$$

subject to the joint fixed point. The induced bimatrix payoffs are

$$U_{ij}^A = \Pi_A(i, j, b(i,j)), \qquad U_{ij}^B = \Pi_B(i, j, b(i,j)). \tag{19}$$

Because both candidate sets are finite, a mixed Nash equilibrium exists. For mixed strategies $(\boldsymbol{\sigma}_A, \boldsymbol{\sigma}_B)$, full-grid regret is

$$\epsilon_A = \max_i \; \boldsymbol{e}_i^\top U^A \boldsymbol{\sigma}_B - \boldsymbol{\sigma}_A^\top U^A \boldsymbol{\sigma}_B, \tag{20}$$

$$\epsilon_B = \max_j \; \boldsymbol{\sigma}_A^\top U^B \boldsymbol{e}_j - \boldsymbol{\sigma}_A^\top U^B \boldsymbol{\sigma}_B. \tag{21}$$

The solver starts from the uniform-game support. Nashpy enumerates restricted equilibria. If support enumeration returns no valid profile, Lemke–Howson labels are used. The algorithm then evaluates all 225 unilateral deviations for each provider, adds profitable best responses, and solves the enlarged game. It stops when $\max(\epsilon_A, \epsilon_B) \le 10^{-7}$. Among multiple restricted equilibria, the implementation selects the profile with the smallest full-grid regret. This selection rule is recorded in the artifact.

The grid certificate is complemented by a Latin-hypercube diagnostic with seed 20260712. Each provider receives 512 off-grid candidates inside the convex hull of Eq. (16). The opponent remains at the finite-game equilibrium. This test can reveal a discretisation problem, but cannot prove a continuous-space equilibrium.

3.6 Data anchors, parameter design, and verification

The demand shape comes from the official BurstGPT trace [17]. The pinned file contains 1,429,737 records. Requests are assigned to eight three-hour periods. The first and last partial days are removed, leaving 59 complete days. Each day is normalised to unit token mass before the period means are computed. The resulting period-share vector is

$$\boldsymbol{\nu} = (0.1116, 0.0273, 0.0132, 0.1198, 0.1761, 0.2169, 0.1679, 0.1672).$$

Both request types use this shape, scaled by their population shares. The source does not state the time zone, so the periods describe relative positions within the recorded day rather than local clock time. The source commit, raw checksum, aggregation code, and derived profile are stored with the artifacts.

The QoS anchor uses controlled vLLM 0.22.0 measurements on one NVIDIA RTX 4090 with 24,564 MiB of memory, driver 596.21, and PyTorch 2.11.0+cu130. Qwen2.5-0.5B-Instruct is tested at concurrency levels 64, 128, 224, 384, and 512 with 512 requests per level. Qwen2.5-3B-Instruct is tested at levels 32, 64, 128, 224, and 384 with 384 requests per level. The fixed prompt is "Explain in two concise sentences why batching can improve GPU inference throughput. Do not use a list." Both runs use a maximum of 128 output tokens, two discarded warm-up requests, seed 42, and five repeats. QoS is the proportion of requests with TTFT no greater than 0.5 seconds. Concurrency 224 is the reference level $u = 1$ because it is the highest shared level with unit QoS. The pooled fit constrains $\bar{u} \geq 1$ and gives $\bar{u} = 1.000$, $\zeta = 0.5747$, and RMSE 0.0561. Leave-one-model-out RMSE is 0.0454 or 0.1046, depending on the training model.

Table 2: Main simulation parameters and their evidence status.

Parameter	Main value	Basis
Periods and period length	$T = 8$, $h = 3$ hours	BurstGPT aggregation
Total native demand	1100 normalised units	Fixed simulation scale
Rigid/flexible population share	0.4/0.6	Synthetic design
Flexible fraction ϕ_R/ϕ_F	0/0.8	Synthetic design
Price sensitivity α_R/α_F	2/5	Synthetic design
Migration cost κ_R/κ_F	2/0.35	Synthetic design
Maximum shift H_R/H_F	0/2 periods	Synthetic design
Provider capacities G_A/G_B	180/72	Congested stress case
QoS threshold and strength	1.000/0.5747	Pooled vLLM fit
QoS choice weight ω_q	1	Synthetic design
Routing QoS weight η	3	Synthetic design
Channel preferences (b_I, b_A, b_B)	$(1.05, 1, 1)$	Synthetic design
Fixed shares when spatial response is off	$(0.12, 0.50, 0.38)$	Decomposition control
Capacity/degradation cost	0.015/0.35	Synthetic design

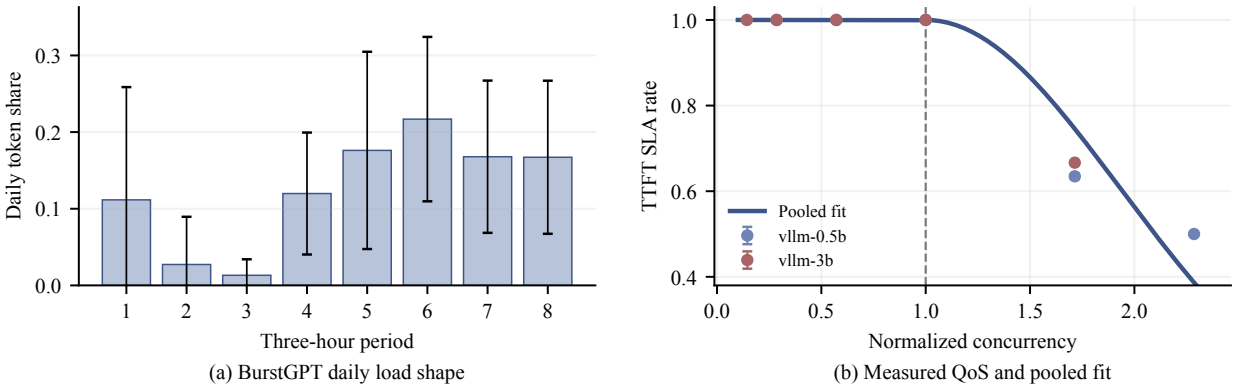
Verification checks are distinct from validation anchors. Automated tests verify OD row conservation, total demand, wholesale transfer cancellation, price-shape signs, fixed-point residuals, finite-game regret, deterministic artifacts, and figure inputs. A candidate is excluded if the joint fixed point fails. Validation is limited to the BurstGPT load shape and the measured QoS shape. Price sensitivity, migration cost, capacity, and cost parameters remain synthetic.

The parameter study re-solves both the uniform and dynamic games for nine cases. Capacity is scaled by 0.85 or 1.15. Price sensitivity is scaled by 0.8 or 1.2. Migration cost is scaled by 0.7 or 1.3. The QoS threshold is shifted by -0.05 or $+0.05$. In every case, the dynamic game scans all 225 candidates for each provider, while the uniform game scans its complete nine-candidate zero-slope restriction.

4 Experimental results

4.1 Load and QoS anchors

Figure 2 reports the two external anchors. BurstGPT token demand is lowest in periods 2 and 3 and highest in period 6. The between-day standard deviations are large, so the trace is used only for the mean daily shape. The vLLM points remain at unit SLA rate up to a normalised utilisation of one and decline after overload. The pooled curve reproduces this threshold pattern but does not represent a production cluster.



(a) BurstGPT daily load shape

(b) Measured QoS and pooled fit

Figure 2: Inputs used to anchor the final simulation. Panel (a) shows the mean and standard deviation of day-normalised BurstGPT token shares across 59 complete days. Panel (b) shows mean controlled vLLM TTFT-SLA measurements across five repeats, 95% confidence intervals, and the pooled threshold-exponential fit.

4.2 Finite-grid equilibrium and main policy effect

The uniform game converged after two double-oracle rounds. Its active provider strategies were identical: $(\bar{w}, \delta^w, \bar{p}^D, \delta^D) = (0.575, 0, 0.6, 0)$. The dynamic game converged after three rounds to a pure finite-grid profile. Provider A used $(0.575, 0.2, 0.6, 0.2)$ and provider B used $(0.575, 0.4, 0.6, 0.4)$. The intermediary selected a retail base of 1.1, zero retail slope, and routing sensitivity 1.5. Time variation therefore came from provider wholesale and direct prices in this case.

Table 3: Uniform and dynamic finite-game equilibrium outcomes.

Metric	Uniform	Dynamic	Change
Aggregate peak load	224.480	195.226	−13.03%
Aggregate peak-to-average ratio	1.633	1.420	−13.03%
Maximum provider utilisation	1.455	1.207	−17.08%
Minimum provider QoS	0.888	0.976	+0.088
Temporally moved fraction	0.313	0.330	+0.0167
Provider A profit	944.852	1039.183	+9.98%
Provider B profit	876.969	958.723	+9.32%
Intermediary profit	150.698	157.170	+4.29%
System profit	1972.519	2155.076	+9.25%

Table 3 and Figure 3 show that the dynamic equilibrium reduced the market peak by 29.254 normalised load units. The largest reduction occurred in period 6. Provider B remained the tighter-capacity provider, but its maximum utilisation fell from 1.455 to 1.207. Its minimum QoS increased from 0.888 to 0.976. Provider A remained below the calibrated degradation threshold in both policies.

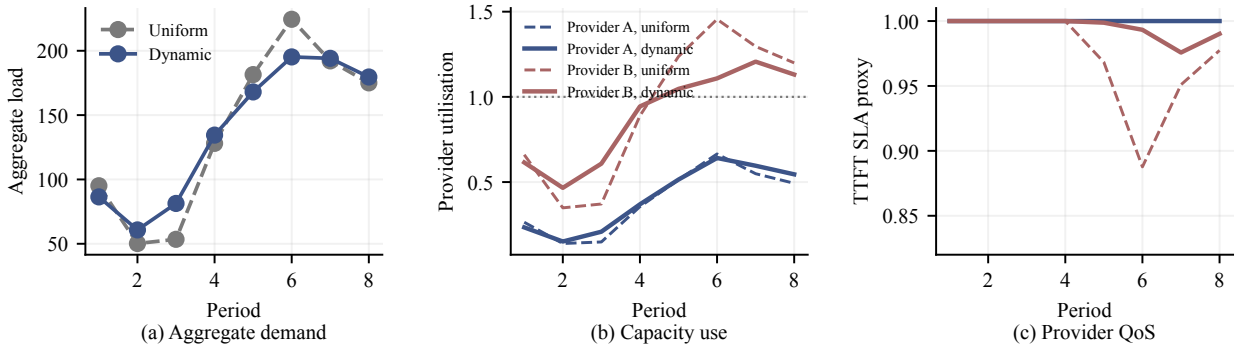


Figure 3: Expected equilibrium profiles under uniform and dynamic pricing. Panel (a) reports aggregate load, panel (b) reports provider utilisation, and panel (c) reports the TTFT-SLA proxy. Solid lines denote the dynamic equilibrium and dashed lines denote the uniform equilibrium in panels (b) and (c).

The demand result is not caused by market exit. Total demand is 1100 under both policies by construction and by artifact check. The flexible-demand centroid changes from 5.304 to 5.197 under the two equilibrium policies. More importantly, the full OD matrix records a temporally moved fraction of 0.330 under dynamic pricing, compared with 0.313 under uniform pricing.

4.3 Numerical equilibrium checks

The dynamic double oracle evaluated 2,445 distinct provider pairs. Full-grid regret fell from 44.284 in round 1 to 20.561 in round 2 and zero in round 3. The uniform game evaluated 56 pairs and reached zero regret in round 2. The maximum joint routing–QoS residual was 6.63×10^{-10} for the dynamic profile and 6.56×10^{-10} for the uniform profile.

The off-grid diagnostic evaluated 512 Latin-hypercube candidates for each provider. The incumbent finite-grid strategy remained the best candidate for both providers, and no sampled candidate produced a positive payoff gain. All sampled fixed points converged, and the largest residual was below 10^{-9}. Figure 4 reports both checks. The right panel shows the ten highest sampled candidates, including the incumbent at rank one.

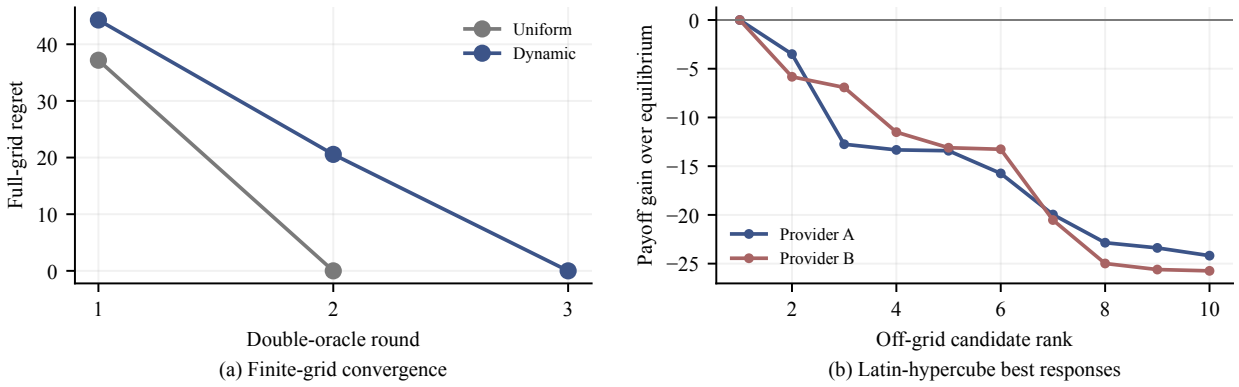


Figure 4: Finite-game and off-grid diagnostics. Panel (a) shows full-grid regret by double-oracle round. Panel (b) shows the payoff difference between the ten highest-ranked off-grid candidates and the finite-grid equilibrium payoff. The zero off-grid result is a finite random-search diagnostic, not a continuous-space proof.

4.4 Temporal and spatial mechanism decomposition

The main comparison contains both time movement and channel switching. To separate them, provider and intermediary price vectors and the routing sensitivity were fixed at their combined-mechanism equilibrium values. Routing and QoS were then re-solved with temporal response, spatial response, both responses, or neither response.

With dynamic prices, the temporal-only mechanism reduced aggregate peak load by 42.015 and maximum provider utilisation by 0.225. Minimum QoS increased by 0.0698. The spatial-only mechanism left aggregate peak load unchanged, as required by its construction. It reduced maximum provider utilisation by 0.1017 and increased minimum QoS by 0.0377. The combined mechanism reduced aggregate peak load by 43.391, reduced maximum provider utilisation by 0.1902, and increased minimum QoS by 0.0623.

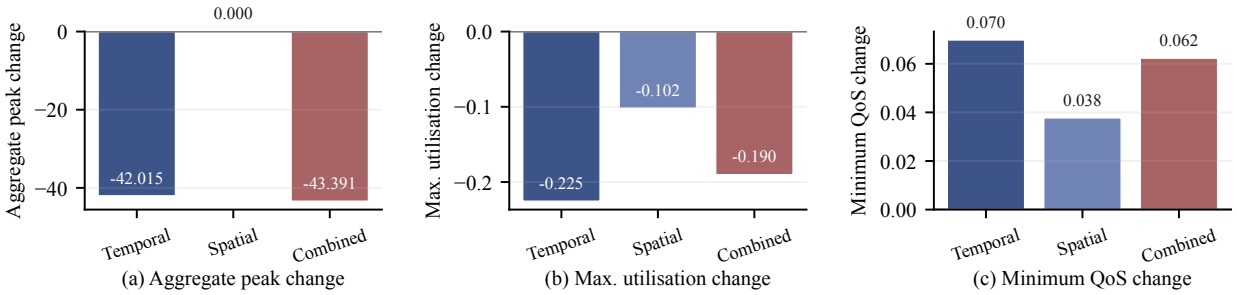


Figure 5: Fixed-policy mechanism decomposition for the dynamic equilibrium prices. Changes are measured against the same prices with temporal and spatial responses disabled. Temporal response changes the aggregate profile. Spatial response reallocates demand across channels and providers while preserving each period's aggregate demand.

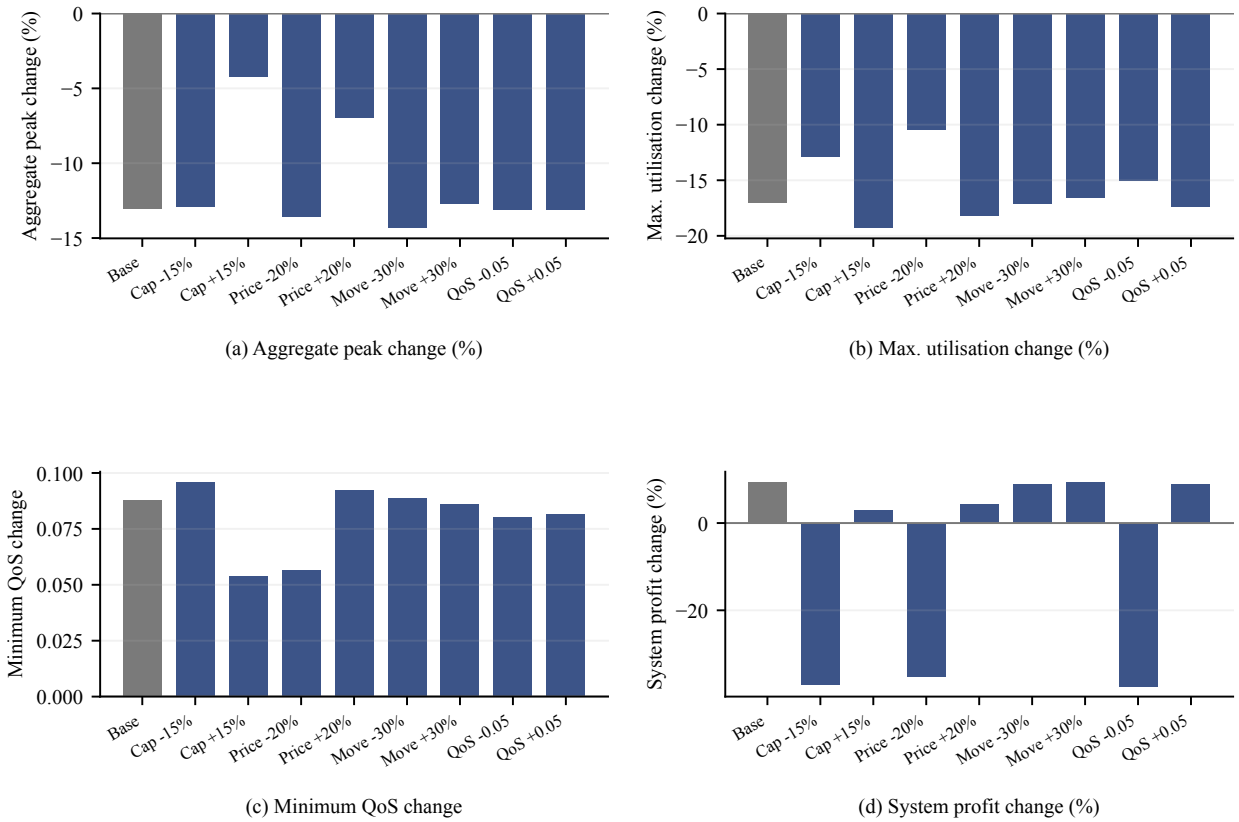
Figure 5 confirms that the two mechanisms are distinct. It also shows that their combined outcome is not additive. QoS feeds back into both channel choice and routing, so enabling one response changes the operating point of the other.

4.5 Fully re-solved sensitivity and profit variation

All nine parameter cases reached zero regret on their complete finite grids. Every joint residual was below 9.88×10^{-10}. Dynamic pricing reduced aggregate peak load by 4.23–14.31% and maximum provider utilisation by 10.49–19.27%. Minimum QoS increased by 0.0540–0.0958. These directions held under lower and higher capacity, price sensitivity, migration cost, and QoS threshold.

The magnitude was not monotone in every parameter. Higher capacity produced the smallest aggregate peak reduction, 4.23%, because congestion pressure was weaker. Lower migration cost produced the largest reduction, 14.31%. Higher price sensitivity reduced the peak by 6.96%, while lower sensitivity reduced it by 13.56%. The equilibrium price shapes changed across these cases, so a one-parameter behavioural explanation would be incomplete.

The profit result changed sign. System profit changed by −37.43% to +9.41% across the nine re-solves. The sign was negative under low capacity, low price sensitivity, and a lower QoS threshold. It was positive in the other six cases. Figure 6 displays this contrast. Peak and QoS results keep the same direction across the tested cases, whereas profit does not.



(a) Aggregate peak change (%)

(b) Max. utilisation change (%)

(c) Minimum QoS change

(d) System profit change (%)

Figure 6: Results from nine fully re-solved cases. Each case recomputes the uniform and dynamic equilibria, scans all 225 dynamic-price candidates for each provider, and scans the complete nine-candidate uniform restriction. Panels report the dynamic-minus-uniform change in aggregate peak, maximum provider utilisation, minimum QoS, and system profit.

4.6 Interpretation and evidence limits

The temporal result is comparable in purpose, but not in physical mechanism, to electricity demand response. Price shapes move flexible requests away from the observed load peak. The spatial result has no direct counterpart in a single-utility electricity model. Users and intermediary traffic can move between providers, which relieves a provider hotspot even when the market peak is unchanged.

The model is a mechanism study rather than a production forecast. BurstGPT calibrates only the mean intraday load shape, and day-to-day load uncertainty is not propagated through the equilibrium. The vLLM experiments calibrate only a simple single-GPU QoS curve. Price sensitivity, migration cost, capacities, and costs are synthetic. The fixed-demand design also omits market expansion and exit. This restriction makes temporal conservation identifiable, but it limits welfare interpretation.

The equilibrium statement is similarly bounded. Zero regret establishes a Nash equilibrium of the declared finite candidate game. The Latin-hypercube result reduces concern about nearby discretisation error, but it does not establish a continuous-space equilibrium. The pure profile also depends on the finite intermediary response set. A broader continuous intermediary optimisation could change provider payoffs.

5 Conclusion and outlook

This paper developed a conserved-demand simulation of time-of-use pricing in a fixed-capacity inference market. The model separates time movement from provider and channel substitution. It also keeps routing, QoS, payment, and provider competition in one accounting system.

The finite-game results support a limited but consistent service-quality finding. Dynamic pricing reduced aggregate peak load by 13.03%, reduced the largest provider utilisation by 17.08%, and raised minimum QoS from 0.888 to 0.976. The same directions held in nine fully re-solved parameter cases. Mechanism decomposition showed that temporal response lowered the market peak, while spatial response relieved a provider hotspot without changing that peak.

The evidence does not support a general profit claim because profit changed sign across the parameter re-solves. It also does not prove equilibrium in a continuous price space or predict production user behaviour. The conclusions are restricted to the stated finite game and calibration design.

Future work should estimate price sensitivity and migration cost from controlled user or workload experiments. It should test longer contexts, additional models, and multi-GPU serving systems. A larger market with several intermediaries would allow competition in routing as well as pricing. Continuous best-response optimisation and formal uniqueness conditions for the joint fixed point would strengthen the equilibrium analysis.

Data and code availability

The code, configuration, derived BurstGPT profile, machine-readable result files, and figure sources are organised in the public repository at https://github.com/cccht/paper_token_price. The raw BurstGPT trace is not redistributed. Its official repository, pinned commit, checksum, licence, and aggregation procedure are recorded with the derived data. The final simulation artifacts include source-file hashes, the Git commit, the dirty-worktree flag, solver settings, and executed commands.

Declaration of generative AI and AI-assisted technologies in manuscript preparation

During preparation of this work, the authors used OpenAI Codex for language editing, consistency checks, internal reviewer-style checks, code testing, and LaTeX layout checks. The authors reviewed and edited all generated material and take full responsibility for the article. Figure 1 was created as editable Draw.io vector artwork using Streamline icons. No generative image model was used to create or alter that figure.

References

[1] Gyeong-In Yu, Joo Seong Jeong, Geon-Woo Kim, Soojeong Kim, and Byung-Gon Chun. Orca: A distributed serving system for transformer-based generative models. In *16th USENIX Symposium on Operating Systems Design and Implementation (OSDI 22)*, pages 521–538, 2022. URL https://www.usenix.org/conference/osdi22/presentation/yu.

[2] Woosuk Kwon, Zhuohan Li, Siyuan Zhuang, Ying Sheng, Lianmin Zheng, Cody Hao Yu, Joseph E. Gonzalez, Hao Zhang, and Ion Stoica. Efficient memory management for large language model serving with PagedAttention. In *Proceedings of the 29th Symposium on Operating Systems Principles*, pages 611–626, 2023. doi: 10.1145/3600006.3613165. URL https://doi.org/10.1145/3600006.3613165.

[3] Amey Agrawal, Nitin Kedia, Ashish Panwar, Jayashree Mohan, Nipun Kwatra, Bhargav Gulavani, Alexey Tumanov, and Ramachandran Ramjee. Taming Throughput-Latency tradeoff in LLM inference with Sarathi-Serve. In *18th USENIX Symposium on Operating Systems Design and Implementation (OSDI 24)*, pages 117–134. USENIX Association, 2024. URL https://www.usenix.org/conference/osdi24/presentation/agrawal.

[4] Pratyush Patel, Esha Choukse, Chaojie Zhang, Aashaka Shah, Íñigo Goiri, Saeed Maleki, and Ricardo Bianchini. Splitwise: Efficient generative LLM inference using phase splitting. In *2024 ACM/IEEE 51st Annual International Symposium on Computer Architecture (ISCA)*, pages 118–132, 2024. doi: 10.1109/ISCA59077.2024.00019. URL https://doi.org/10.1109/ISCA59077.2024.00019.

[5] Yinmin Zhong, Shengyu Liu, Junda Chen, Jianbo Hu, Yibo Zhu, Xuanzhe Liu, Xin Jin, and Hao Zhang. DistServe: Disaggregating prefill and decoding for goodput-optimized large language model serving. In *18th USENIX Symposium on Operating Systems Design and Implementation (OSDI 24)*, 2024. URL https://www.usenix.org/conference/osdi24/presentation/zhong-yinmin.

[6] Fred C. Schweppe, Michael C. Caramanis, Richard D. Tabors, and Roger E. Bohn. *Spot Pricing of Electricity*. Springer, 1988. doi: 10.1007/978-1-4613-1683-1. URL https://doi.org/10.1007/978-1-4613-1683-1.

[7] Severin Borenstein. The long-run efficiency of real-time electricity pricing. *Energy Journal*, 26(3):93–116, 2005. doi: 10.5547/ISSN0195-6574-EJ-Vol26-No3-5. URL https://doi.org/10.5547/ISSN0195-6574-EJ-Vol26-No3-5.

[8] M. H. Albadi and E. F. El-Saadany. A summary of demand response in electricity markets. *Electric Power Systems Research*, 78(11):1989–1996, 2008. doi: 10.1016/j.epsr.2008.04.002. URL https://doi.org/10.1016/j.epsr.2008.04.002.

[9] Ahmad Faruqui and Sanem Sergici. Household response to dynamic pricing of electricity: A survey of 15 experiments. *Journal of Regulatory Economics*, 38:193–225, 2010. doi: 10.1007/s11149-010-9127-y. URL https://doi.org/10.1007/s11149-010-9127-y.

[10] Pierluigi Siano. Demand response and smart grids—a survey. *Renewable and Sustainable Energy Reviews*, 30:461–478, 2014. doi: 10.1016/j.rser.2013.10.022. URL https://doi.org/10.1016/j.rser.2013.10.022.

[11] Mert Demirer, Andrey Fradkin, Nadav Tadelis, and Sida Peng. The emerging market for intelligence: Pricing, supply, and demand for LLMs. Technical Report 34608, National Bureau of Economic Research, 2025. URL https://www.nber.org/papers/w34608.

[12] Dirk Bergemann, Alessandro Bonatti, and Alex Smolin. Menu pricing of large language models. *arXiv preprint arXiv:2502.07736*, 2025. URL https://arxiv.org/abs/2502.07736.

[13] Zhendong Guo, Wenchao Bai, and Jiahui Jin. PriLLM: Pricing online LLM services with data-calibrated stackelberg routing game. *Proceedings of the AAAI Conference on Artificial Intelligence*, 40(20):17005–17013, 2026. doi: 10.1609/aaai.v40i20.38748. URL https://doi.org/10.1609/aaai.v40i20.38748.

[14] Junji Yan, Asrin Efe Yorulmaz, Hanchen Zhou, and Tamer Başar. A stackelberg game framework with drainability guardrails for pricing and scaling in multi-tenant GPU cloud platforms. *arXiv preprint arXiv:2604.16802*, 2026. URL https://arxiv.org/abs/2604.16802.

[15] William J. Cunningham. Token management in multi-tenant AI inference platforms. *arXiv preprint arXiv:2603.00356*, 2026. URL https://arxiv.org/abs/2603.00356.

[16] Amey Agrawal, Nitin Kedia, Jayashree Mohan, Ashish Panwar, Nipun Kwatra, Bhargav Gulavani, Ramachandran Ramjee, and Alexey Tumanov. Vidur: A large-scale simulation framework for LLM inference. In *Proceedings of Machine Learning and Systems*, volume 6, 2024. URL https://proceedings.mlsys.org/paper_files/paper/2024/hash/b74a8de47d2b3c928360e0a011f48351-Abstract-Conference.html.

[17] Yuxin Wang, Yuhan Chen, Zeyu Li, Xueze Kang, Yuchu Fang, Yeju Zhou, Yang Zheng, Zhenheng Tang, Xin He, Rui Guo, Xin Wang, Qiang Wang, Amelie Chi Zhou, and Xiaowen Chu. BurstGPT: A real-world workload dataset to optimize LLM serving systems. In *Proceedings of the 31st ACM SIGKDD Conference on Knowledge Discovery and Data Mining V.2*, pages 5831–5841. ACM, 2025. doi: 10.1145/3711896.3737413. URL https://doi.org/10.1145/3711896.3737413.

[18] Daniel Crankshaw, Xin Wang, Guilio Zhou, Michael J. Franklin, Joseph E. Gonzalez, and Ion Stoica. Clipper: A low-latency online prediction serving system. In *14th USENIX Symposium on Networked Systems Design and Implementation (NSDI 17)*, pages 613–627, 2017. URL https://www.usenix.org/conference/nsdi17/technical-sessions/presentation/crankshaw.

[19] Arpan Gujarati, Reza Karimi, Safya Alzayat, Wei Hao, Antoine Kaufmann, Ymir Vigfusson, and Jonathan Mace. Serving DNNs like clockwork: Performance predictability from the bottom up. In *14th USENIX Symposium on Operating Systems Design and Implementation (OSDI 20)*, 2020. URL https://www.usenix.org/conference/osdi20/presentation/gujarati.

[20] Hunt Allcott. Rethinking real-time electricity pricing. *Resource and Energy Economics*, 33 (4):820–842, 2011. doi: 10.1016/j.reseneeco.2011.06.003. URL https://doi.org/10.1016/j.reseneeco.2011.06.003.

[21] Pedram Samadi, Amir-Hamed Mohsenian-Rad, Robert Schober, Vincent W. S. Wong, and Juri Jatskevich. Optimal real-time pricing algorithm based on utility maximization for smart grid. In *2010 First IEEE International Conference on Smart Grid Communications*, pages 415–420, 2010. doi: 10.1109/SMARTGRID.2010.5622077. URL https://doi.org/10.1109/SMARTGRID.2010.5622077.

[22] Peng Yang, Gongguo Tang, and Arye Nehorai. A game-theoretic approach for optimal time-of-use electricity pricing. *IEEE Transactions on Power Systems*, 28(2):884–892, 2013. doi: 10.1109/TPWRS.2012.2207134. URL https://doi.org/10.1109/TPWRS.2012.2207134.

[23] Mengmeng Yu and Seung Ho Hong. Supply–demand balancing for power management in smart grid: A stackelberg game approach. *Applied Energy*, 164:702–710, 2016. doi: 10.1016/j.apenergy.2015.12.039. URL https://doi.org/10.1016/j.apenergy.2015.12.039.

[24] Dipti Srinivasan, Sanjana Rajgarhia, Bharat Menon Radhakrishnan, Anurag Sharma, and H. P. Khincha. Game-theory based dynamic pricing strategies for demand side management in smart grids. *Energy*, 126:132–143, 2017. doi: 10.1016/j.energy.2016.11.142. URL https://doi.org/10.1016/j.energy.2016.11.142.

[25] William S. Vickrey. Congestion theory and transport investment. *American Economic Review*, 59(2):251–260, 1969. URL https://www.jstor.org/stable/1823678.

[26] Richard Arnott, André de Palma, and Robin Lindsey. Economics of a bottleneck. *Journal of Urban Economics*, 27(1):111–130, 1990. doi: 10.1016/0094-1190(90)90028-L. URL https://doi.org/10.1016/0094-1190(90)90028-L.

[27] Guillermo Gallego and Garrett van Ryzin. Optimal dynamic pricing of inventories with stochastic demand over finite horizons. *Management Science*, 40(8):999–1020, 1994. doi: 10.1287/mnsc.40.8.999. URL https://doi.org/10.1287/mnsc.40.8.999.

[28] Wedad Elmaghraby and Pınar Keskinocak. Dynamic pricing in the presence of inventory considerations: Research overview, current practices, and future directions. *Management Science*, 49(10):1287–1309, 2003. doi: 10.1287/mnsc.49.10.1287.17315. URL https://doi.org/10.1287/mnsc.49.10.1287.17315.

[29] Daniel McFadden. Conditional logit analysis of qualitative choice behavior. In Paul Zarembka, editor, *Frontiers in Econometrics*, pages 105–142. Academic Press, 1974.

[30] Simon P. Anderson, André de Palma, and Jacques-François Thisse. *Discrete Choice Theory of Product Differentiation*. MIT Press, 1992.

[31] Kenneth E. Train. *Discrete Choice Methods with Simulation*. Cambridge University Press, 2 edition, 2009.

[32] Jean-Charles Rochet and Jean Tirole. Platform competition in two-sided markets. *Journal of the European Economic Association*, 1(4):990–1029, 2003. doi: 10.1162/154247603322493212. URL https://doi.org/10.1162/154247603322493212.

[33] John Nash. Non-cooperative games. *Annals of Mathematics*, 54(2):286–295, 1951. doi: 10.2307/1969529. URL https://doi.org/10.2307/1969529.

[34] Drew Fudenberg and Jean Tirole. *Game Theory*. MIT Press, 1991.

[35] Carlton E. Lemke and Joseph T. Howson Jr. Equilibrium points of bimatrix games. *Journal of the Society for Industrial and Applied Mathematics*, 12(2):413–423, 1964. doi: 10.1137/0112033. URL https://doi.org/10.1137/0112033.

[36] Vincent Knight and James Campbell. Nashpy: A python library for the computation of Nash equilibria. *Journal of Open Source Software*, 3(30):904, 2018. doi: 10.21105/joss.00904. URL https://doi.org/10.21105/joss.00904.